

# DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

## UNIT II

### CONTENT-BASED RECOMMENDATION SYSTEMS

### UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

6

High-level architecture of content-based systems - Item profiles, Representing item profiles, Methods for learning user profiles, Similarity-based retrieval, and Classification algorithms.

#### Suggested Activities:

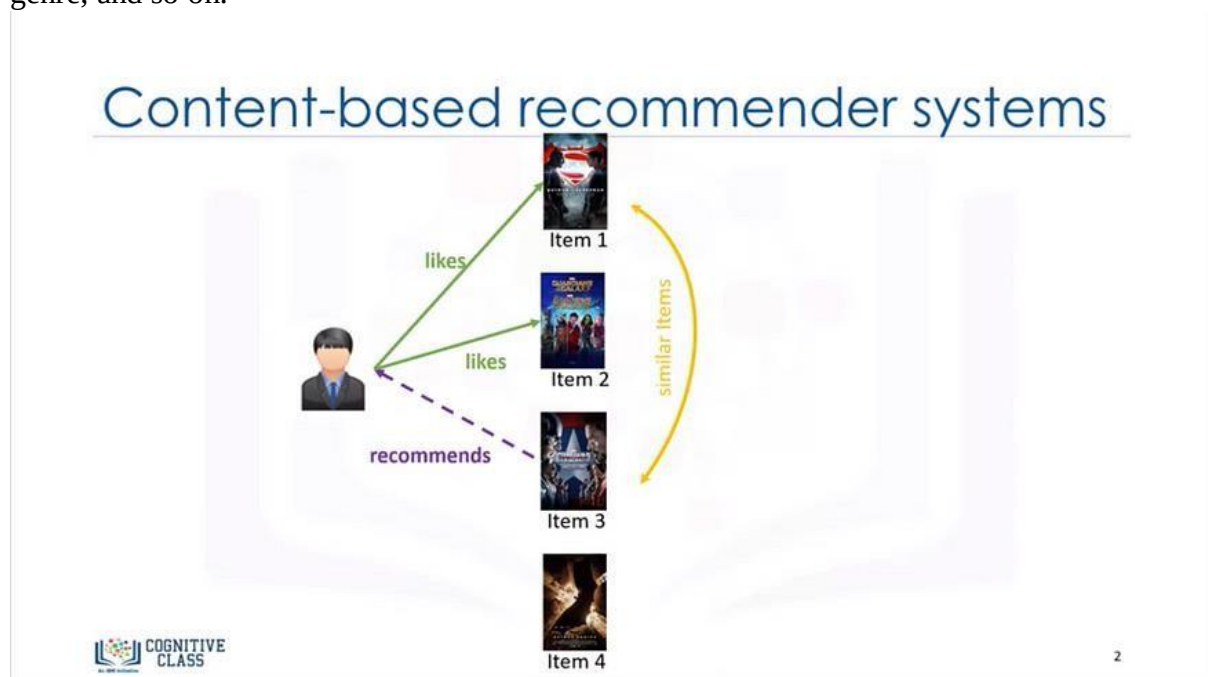
- Assignment on content-based recommendation systems
- Assignment of learning user profiles

#### Suggested Evaluation Methods:

- Quiz on similarity-based retrieval.
- Quiz of content-based filtering

Content-based recommendation system

A *Content-based recommendation system* tries to recommend items to users based on their *profile*. The user's profile revolves around that user's preferences and tastes. It is shaped based on user ratings, including the number of times that user has clicked on different items or perhaps even liked those items. The recommendation process is based on the similarity between those items. Similarity or closeness of items is measured based on the similarity in the content of those items. When we say content, we're talking about things like the items category, tag, genre, and so on.



For example, if we have four movies, and if the user likes or rates the first two items, and if Item 3 is similar to Item 1 in terms of their genre, the engine will also recommend Item 3 to the user. In essence, this is what content-based recommender system engines do. Now, let's dive into a content-based recommender system to see how it works.

## Content-based recommender systems



Let's assume we have a data set of only six movies. This data set shows movies that our user has watched and also the genre of each of the movies. For example, Batman versus Superman is in the Adventure, Super Hero genre and Guardians of the Galaxy is in the Comedy, Adventure, Super Hero and Science-fiction genres. Let's say the user has watched and rated three movies so far and she has given a rating of two out of 10 to the first movie, 10 out of 10 to the second movie and eight out of 10 to the third. The task of the recommender engine is to recommend one of the three candidate movies to this user, or in other, words we want to predict what the user's possible rating would be of the three candidate movies if she were to watch them. To achieve this, we have to build the user profile.

## Weighing the genres

# Weighing the genres



|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
|  | 2  |
|  | 10 |
|  | 8  |

Input User Ratings

X

|                                                                                     | Comedy | Adventure | Super Hero | Sci-Fi |
|-------------------------------------------------------------------------------------|--------|-----------|------------|--------|
|  | 0      | 1         | 1          | 0      |
|  | 1      | 1         | 1          | 1      |
|  | 1      | 0         | 1          | 0      |

Movies Matrix

=

|                                                                                     | Comedy | Adventure | Super Hero | Sci-Fi |
|-------------------------------------------------------------------------------------|--------|-----------|------------|--------|
|  | 0      | 2         | 2          | 0      |
|  | 10     | 10        | 10         | 10     |
|  | 8      | 0         | 8          | 0      |



|  | Comedy | Adventure | Super Hero | Sci-Fi |
|--|--------|-----------|------------|--------|
|  | 0.3    | 0.2       | 0.33       | 0.16   |

User Profile

5



First, we create a vector to show the user's ratings for the movies that she's already watched. We call it Input User Ratings. Then, we encode the movies through the one-hot encoding approach. Genre of movies are used here as a feature set. We use the first three movies to make

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

this matrix, which represents the movie feature set matrix. If we multiply these two matrices we can get the weighted feature set for the movies. Let's take a look at the result. This matrix is also called the Weighted Genre matrix and represents the interests of the user for each genre based on the movies that she's watched. Now, given the Weighted Genre Matrix, we can shape the profile of our active user. Essentially, we can aggregate the weighted genres and then normalize them to find the user profile. It clearly indicates that she likes superhero movies more than other genres.

### Candidate movies for recommendation

|                 | Comedy | Adventure | Super Hero | Sci-Fi |
|-----------------|--------|-----------|------------|--------|
| Fitcher         | 1      | 1         | 0          | 1      |
| The Dark Knight | 0      | 0         | 1          | 0      |
| Spider-Man      | 1      | 0         | 1          | 0      |

COGNITIVE CLASS

6

We use this profile to figure out what movie is proper to recommend to this user. Recall that we also had three candidate movies for recommendation that haven't been watched by the user, we encode these movies as well. Now we're in the position where we have to figure out which of them is most suited to be recommended to the user.

### Finding the recommendation

|              | Comedy | Adventure | Super Hero | Sci-Fi |
|--------------|--------|-----------|------------|--------|
| User Profile | 0.3    | 0.2       | 0.33       | 0.16   |

↓

|                 | Comedy | Adventure | Super Hero | Sci-Fi |
|-----------------|--------|-----------|------------|--------|
| Fitcher         | 1      | 1         | 0          | 1      |
| The Dark Knight | 0      | 0         | 1          | 0      |
| Spider-Man      | 1      | 0         | 1          | 0      |

↓

|                 | Comedy | Adventure | Super Hero | Sci-Fi |
|-----------------|--------|-----------|------------|--------|
| Fitcher         | 0.3    | 0.2       | 0          | 0.16   |
| The Dark Knight | 0      | 0         | 0.33       | 0      |
| Spider-Man      | 0.3    | 0         | 0.33       | 0      |

↓

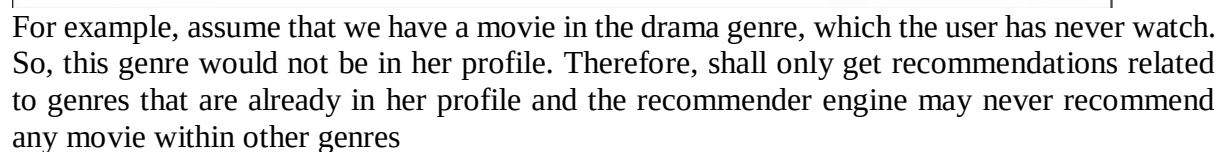
|                 | Weighted Average |
|-----------------|------------------|
| Fitcher         | 0.66             |
| The Dark Knight | 0.33             |
| Spider-Man      | 0.63             |

COGNITIVE CLASS

7

To do this, we simply multiply the User Profile matrix by the candidate Movie Matrix, which results in the Weighted Movies Matrix. It shows the weight of each genre with respect to the User Profile. Now, if we aggregate these weighted ratings, we get the active user's possible

interest level in these three movies. In essence, it's our recommendation lists, which we can sort to rank the movies and recommend them to the user. For example, we can say that the Hitchhiker's Guide to the Galaxy has the highest score in our list, and it's proper to recommend to the user. Now, you can come back and fill the predicted ratings for the user. *So, to recap what we've discussed so far, the recommendation in a content-based system is based on user's taste and the content or feature set items. Such a model is very efficient. However, in some cases, it doesn't work.*



The diagram illustrates the active user feedback loop in a recommender system. It shows the flow of data and components involved in generating recommendations and receiving feedback from an active user.

**Components and Data Flow:**

- Information Source** (oval) provides **Item Descriptions** (stack of rectangles) to the **CONTENT ANALYZER**.
- CONTENT ANALYZER** (rounded rectangle) outputs **Structured Item Representation** (stack of rectangles) to **Represented Items** (cylinder).
- Represented Items** provides input to the **PROFILE LEARNER**.
- User  $u_a$  training examples** (stack of rectangles) also provide input to the **PROFILE LEARNER**.
- The **PROFILE LEARNER** (rounded rectangle) outputs a **User  $u_a$  Profile** (stack of rectangles) to the **PROFILES** cylinder.
- PROFILES** also receives **New Items** (stack of rectangles) as input.
- The **PROFILE LEARNER** outputs **User  $u_a$  feedback** (stack of rectangles) to the **Feedback** cylinder.
- The **Feedback** cylinder provides input to the **Active user  $u_a$**  (oval).
- The **Active user  $u_a$**  provides **User  $u_a$  feedback** (stack of rectangles) back to the **Feedback** cylinder.
- The **Active user  $u_a$**  also provides input to the **FILTERING COMPONENT**.
- The **PROFILES** cylinder provides input to the **FILTERING COMPONENT**.
- The **FILTERING COMPONENT** (rounded rectangle) outputs a **List of recommendations** (stack of rectangles).

**Feedback Loop:** The **Active user  $u_a$**  interacts with the system, providing feedback that is used by the **PROFILE LEARNER** to refine the **User  $u_a$  Profile** and generate a **List of recommendations**.

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

Content-based Information Filtering (IF) systems need proper techniques for representing the items and producing the user profile, and some strategies for comparing the user profile with the item representation.

The recommendation process is performed in three steps, each of which is handled by a separate component:

- **CONTENT ANALYZER** – When information has no structure (e.g. text), some kind of pre-processing step is needed to extract structured relevant information.

The main responsibility of the component is to represent the content of items (e.g. documents, Web pages, news, product descriptions, etc.) coming from information sources in a form suitable for the next processing steps. Data items are analyzed by feature extraction techniques in order to shift item representation from the original information space to the target one (e.g. Web pages represented as keyword vectors).

This representation is the input to the PROFILE LEARNER and FILTERING COMPONENT;

- **PROFILE LEARNER** – This module collects data representative of the user preferences and tries to generalize this data, in order to construct the user profile. Usually, the generalization strategy is realized through machine learning techniques, which are able to infer a model of user interests starting from items liked or disliked in the past. For instance, the PROFILE LEARNER of a Web page recommender can implement a relevance feedback method in which the learning technique combines vectors of positive and negative examples into a prototype vector representing the user profile. Training examples are Web pages on which a positive or negative feedback has been provided by the user;
- **FILTERING COMPONENT** – This module exploits the user profile to suggest relevant items by matching the profile representation against that of items to be recommended. The result is a binary or continuous relevance judgment (computed using some similarity metrics), the latter case resulting in a ranked list of potentially interesting items. In the above-mentioned example, the matching is realized by computing the cosine similarity between the prototype vector and the item vectors.
- The first step of the recommendation process is the one performed by the CONTENT ANALYZER, that usually borrows techniques from Information Retrieval systems. Item descriptions coming from Information Source are processed by the CONTENT ANALYZER, that extracts features (keywords, n-grams, concepts, . . .) from unstructured text to produce a structured item representation, stored in the repository Represented Items.
- In order to construct and update the profile of the active user  $u_a$  (user for which recommendations must be provided) her reactions to items are collected in some way and recorded in the repository Feedback. These reactions, **called annotations or feedback**, together with the related item descriptions, are exploited during the process of learning a model useful to predict the actual relevance of newly presented items. Users can also explicitly define their areas of interest as an initial profile without providing any feedback.
- **Typically, it is possible to distinguish between two kinds of relevance feedback:** positive information (inferring features liked by the user) and negative information (i.e., inferring features the user is not interested in).
- Two different techniques can be adopted for recording user's feedback. When a system requires the user to explicitly evaluate items, this technique is usually referred to as "explicit feedback"; the other technique, called "implicit feedback", does not require any active user involvement, in the sense that feedback is derived from monitoring and



## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

analyzing user's activities. Explicit evaluations indicate how relevant or interesting an item is to the user.

- **There are three main approaches to get explicit relevance feedback:**
- **like/dislike** – items are classified as “relevant” or “not relevant” by adopting a simple binary rating scale
- **ratings** – a discrete numeric scale is usually adopted to judge items.
- **text comments** – Comments about a single item are collected and presented to the users as a means of facilitating the decision-making process. For instance, customer's feedback at Amazon.com or eBay.com might help users in deciding whether an item has been appreciated by the community. Textual comments are helpful, but they can overload the active user because she must read and interpret each comment to decide if it is positive or negative, and to what degree.
- Explicit feedback has the advantage of simplicity, albeit the adoption of numeric/symbolic scales increases the cognitive load on the user, and may not be adequate for catching user's feeling about items. Implicit feedback methods are based on assigning a relevance score to specific user actions on an item, such as saving, discarding, printing, bookmarking, etc. The main advantage is that they do not require a direct user involvement, even though biasing is likely to occur, e.g. interruption of phone calls while reading.
- In order to build the profile of the active user  $u_a$  the training set  $TR_a$  for  $u_a$  must be defined.  $TR_a$  is a set of pairs  $(l_k, r_k)$  where  $r_k$  is the rating provided by  $u_a$  on the item representation  $l_k$
- Given a set of item representation labelled with ratings, the PROFILE LEARNER applies supervised learning algorithms to generate a predictive model – the user profile – which is usually stored in a profile repository for later use by the FILTERING COMPONENT. Given a new item representation, the FILTERING COMPONENT predicts whether it is likely to be of interest for the active user, by comparing features in the item representation to those in the representation of user preferences (stored in the user profile).
- Usually, the FILTERING COMPONENT implements some strategies to rank potentially interesting items according to the relevance with respect to the user profile. Top-ranked items are included in a list of recommendations  $L_a$ , that is presented to  $u_a$ . User tastes usually change in time, therefore up-to-date information must be maintained and provided to the PROFILE LEARNER in order to automatically update the user profile. Further feedback is gathered on generated recommendations by letting users state their satisfaction or dissatisfaction with items in  $L_a$ .
- After gathering that feedback, the learning process is performed again on the new training set, and the resulting profile is adapted to the updated user interests. The iteration of the feedback-learning cycle over time allows the system to take into account the dynamic nature of user preferences.

### Item profiles in content-based RS

In a content-based recommender system, item profiles play a crucial role in representing and describing the characteristics of items within the system. These profiles are used to match the preferences or requirements of users with the features of items, enabling the system to generate personalized recommendations.

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

Here's a breakdown of item profiles in a content-based recommender system:

### 1. Definition:

- **Item Profiles:** Item profiles are representations of items in the recommender system based on their content or features. These features could include attributes, keywords, metadata, or any other relevant information that describes the characteristics of the items.

### 2. Content Representation:

- **Attributes and Features:** Item profiles encompass various attributes and features that define the content of the items. For example, in a movie recommender system, attributes might include genres, actors, directors, release year, and user ratings.

### 3. Creation of Item Profiles:

- **Data Collection:** The creation of item profiles involves collecting and processing data about each item in the system. This could involve extracting information from textual descriptions, analyzing images or audio, or utilizing any available metadata.

### 4. Feature Extraction:

- **Textual Analysis:** For textual content, natural language processing (NLP) techniques may be applied to extract keywords or topics. Image or audio content may involve feature extraction methods to represent visual or auditory characteristics.

### 5. Representation Format:

- **Vectors or Matrices:** Item profiles are often represented as feature vectors or matrices, where each element corresponds to a specific feature or attribute. This representation allows for efficient computation of item similarities and recommendations.

### 6. Similarity Computation:

- **Matching User Preferences:** The similarity between item profiles and user profiles is computed to identify items that closely match the preferences of the user. Similarity metrics like cosine similarity or Euclidean distance may be used for this purpose.

### 7. Dynamic Updating:

- **Adaptation to Changes:** Item profiles may need to be dynamically updated as new items are added to the system or as existing items are modified. This ensures that the recommender system remains accurate and up-to-date.

### 8. Example in Movie Recommendations:



## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

- **Attributes:** In a movie recommender system, item profiles could include attributes such as genre (e.g., action, comedy, drama), director, actors, release year, and user ratings.
- **Profile Representation:** Each movie's profile is represented as a vector where each element corresponds to a feature, for example, [0.7, 0.4, 0.2, 0.9, ...] for a particular movie.

### 9. Integration with User Profiles:

- **Collaborative Filtering:** Item profiles are often used in conjunction with user profiles in hybrid recommender systems. Collaborative filtering techniques may leverage both user-item interactions and content-based item profiles for more accurate recommendations.

### Representing item profiles in content-based Recommender system

In a content-based recommender system, item profiles are representations of items based on their features or attributes. These representations are crucial for matching the content of items with user preferences. Here are common methods for representing item profiles in a content-based recommender system:

#### Feature Vector Representation:

**Definition:** Represent each item as a feature vector, where each element of the vector corresponds to a specific feature or attribute.

**Example:** In a movie recommender system, an item profile could be represented as a vector with elements for features such as genre, director, actors, release year, etc.

Vector Example: [0.8, 0.2, 0.5, 2010, ...]

#### TF-IDF Representation (Text Data):

**Definition:** If items have textual descriptions or content, use the Term Frequency-Inverse Document Frequency (TF-IDF) representation.

**Example:** Create a TF-IDF vector for each item based on the occurrence of keywords in its description.

Vector Example: [0.1, 0.5, 0.2, 0.0, ...]

#### Image Feature Extraction (Visual Data):

**Definition:** For items with visual content (e.g., images), use feature extraction techniques to represent images as numerical vectors.

**Example:** Utilize deep learning models to extract features from images, representing each item as a vector.

Vector Example: [0.3, 0.7, 0.1, ...]

#### Audio Feature Extraction (Audio Data):

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

**Definition:** For items with audio content, extract audio features using methods like MFCC (Mel-Frequency Cepstral Coefficients).

**Example:** Represent each item with a vector based on its audio features.

**Vector Example:** [0.2, 0.6, 0.4, ...]

### **Metadata Representation:**

**Definition:** Utilize metadata associated with items to create item profiles.

**Example:** If items have metadata such as tags, categories, or user-assigned labels, represent them as elements in the item profile.

**Vector Example:** [1, 0, 1, 0, ...] (binary representation of metadata presence)

### **Hybrid Representations:**

**Definition:** Combine multiple types of representations to create hybrid item profiles.

**Example:** Combine textual features, image features, and metadata into a single feature vector for each item.

**Vector Example:** [0.2, 0.4, 0.1, 0.0, 0.8, ...]

### **Normalization and Scaling:**

**Definition:** Normalize or scale the values in the item profile vectors to ensure consistency and avoid bias due to different scales.

**Example:** Scale values to be in the range [0, 1] or use z-score normalization.

### **Dynamic Updating:**

**Definition:** Regularly update item profiles as new items are added or existing items are modified.

**Example:** Recompute feature vectors when new data becomes available or when existing data is updated.

### **Sparse Representation:**

**Definition:** Represent item profiles as sparse vectors, especially when dealing with high-dimensional data where most features are zero.

**Example:** Use sparse matrix representations for efficient storage and computation.

### **Embedding Models (e.g., Word Embeddings):**

**Definition:** Utilize pre-trained embeddings or train embeddings for categorical features to represent items.

**Example:** Embed categorical features like genres or directors into continuous vector spaces.

**Vector Example:** [0.6, -0.2, 0.8, ...]

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

The choice of representation method depends on the nature of the data and features associated with items in your recommender system. Experimentation and evaluation are crucial to determining the most effective representation for your specific use case.

### **Methods for learning user profiles in content-based Recommender system**

In content-based recommender systems, learning user profiles involves creating and updating representations of user preferences based on their interactions with items and the content characteristics of those items. Here are some common methods for learning user profiles in content-based recommender systems:

#### **1. Term Frequency-Inverse Document Frequency (TF-IDF):**

- Utilizes a numerical statistic that reflects the importance of a term in a document relative to a collection of documents. TF-IDF is often used to represent the content characteristics of items and can be aggregated to create user profiles based on their interactions.

#### **2. Keyword Extraction:**

- Identifies and extracts important keywords from textual content associated with items and uses these keywords to build user profiles. Techniques like Natural Language Processing (NLP) and text mining can be employed for this purpose.

#### **3. Vector Space Model:**

- Represents items and user preferences in a high-dimensional vector space, where the dimensions correspond to different features or characteristics. The user profile is then formed by aggregating the vectors of items the user has interacted with.

#### **4. Machine Learning Algorithms:**

- Utilizes machine learning algorithms to learn user preferences based on historical interactions. Techniques like classification or regression can be employed to predict user preferences for new items.

#### **5. Neural Networks:**

- Employs neural network architectures, such as feedforward neural networks or recurrent neural networks, to automatically learn user profiles from the content features of items and user interactions.

#### **6. Feature Weighting and Selection:**

- Assigns weights to different features or characteristics based on their importance in representing user preferences. Feature selection techniques can be used to identify the most relevant features for constructing user profiles.

#### **7. User Feedback Incorporation:**

- Incorporates explicit or implicit user feedback to adjust and refine user profiles. This feedback may include user ratings, clicks, views, or other relevant signals.

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

### 8. Ontology and Semantic Similarity:

- Utilizes ontologies and semantic similarity measures to represent and measure the semantic relationships between items and user preferences. This approach is particularly useful for handling structured and semantic content.

### 9. Temporal Considerations:

- Takes into account the temporal aspects of user interactions to capture changes in preferences over time. This can involve assigning different weights to items based on recency or considering time-sensitive features.

### 10. Ensemble Methods:

- Combines multiple content-based methods to create a more robust and accurate user profile. Ensemble methods can help mitigate the limitations of individual techniques.

The specific method chosen often depends on the nature of the content, the available data, and the computational resources at hand. Content-based recommender systems aim to capture the intrinsic characteristics of items and users to provide personalized recommendations based on content similarity.

## Similarity-based retrieval

Similarity-based retrieval is a fundamental concept in content-based recommender systems. These systems recommend items to users based on the similarity between items' features and the user's preferences. Here's how similarity-based retrieval works in a content-based recommender system:

- **Feature Extraction:** In a content-based recommender system, items (such as articles, movies, products) are described by a set of features or attributes. These features could include textual content, metadata, user ratings, etc. The first step is to extract relevant features from the items.
- **Feature Representation:** After extracting features, each item is represented as a feature vector. This vector typically contains numerical values representing the item's attributes. For example, if the features include text, techniques like TF-IDF or word embeddings can be used to represent text features as numerical vectors.
- **Similarity Metric:** A similarity metric is chosen to quantify the similarity between two feature vectors. Common similarity metrics include cosine similarity, Euclidean distance, Pearson correlation, and Jaccard similarity, among others. The choice of similarity metric depends on the nature of the features and the problem domain.
- **User Profile Creation:** When a user interacts with the system, their preferences are captured to create a user profile. This profile contains information about the user's preferences, often represented as a feature vector similar to item vectors. The user profile is typically updated over time as the user interacts more with the system.

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

- **Recommendation Generation:** To generate recommendations for a user, the system finds items that are most similar to the user's profile. This is done by calculating the similarity between the user profile vector and the vectors representing all available items. The items with the highest similarity scores are recommended to the user.
- **Ranking and Filtering:** The recommended items may undergo additional ranking and filtering based on factors such as popularity, diversity, or relevance. This step ensures that the most suitable recommendations are presented to the user.
- **Feedback Incorporation:** As users interact with the recommended items, their feedback (e.g., clicks, ratings) is used to update both the user profile and the item features. This feedback loop helps improve the accuracy and relevance of future recommendations.

Overall, similarity-based retrieval plays a crucial role in content-based recommender systems by enabling personalized recommendations based on the similarity between item features and user preferences. When the item selection problem in collaborative filtering can be described as “recommend items that similar users liked”, content-based recommendation is commonly described as “recommend items that are similar to those the user liked in the past”. Therefore, the task for a recommender system is again based on a user profile– to predict, for the items that the current user has not seen, whether he or she will like them. The most common techniques that rely on the vector-space document representation model will be described in this section.

### Nearest Neighbors

At first, straight forward, method for estimating to what extent a certain document will be of interest to a user is simply to check whether the user liked similar documents in the past. To do this, two pieces of information are required.

First, we need some history of “like/dislike” statements made by the user about previous items. Similar to collaborative approaches, these ratings can be provided either explicitly via the user interface or implicitly by monitoring the user’s behavior. Second, a measure is needed that captures the similarity of two documents. In most reported approaches, the cosine similarity measure is used to evaluate whether two document vectors are similar.

The similarity between two items  $a$  and  $b$  – viewed as the corresponding rating vectors  $\vec{a}$  and  $\vec{b}$  – is formally defined as follows:

$$\text{sim}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{|\vec{a}| * |\vec{b}|} \quad (2.5)$$

The  $\cdot$  symbol is the dot product of vectors.  $|\vec{a}|$  is the Euclidian length of the vector, which is defined as the square root of the dot product of the vector with itself.

The prediction for a not-yet-seen item  $d$  is based on letting the  $k$  most similar items for which a rating exists “vote” for  $n$ . If, for instance, four out of  $k = 5$  of the most similar items were liked by the current user, the system may guess that the chance that  $d$  will also be liked is relatively high. Besides varying the neighborhood size  $k$ , several other variations are possible, such as binarization of ratings, using a minimum similarity threshold, or weighting of the votes based on the degree of similarity.

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

Such a k-nearest-neighbour method (kNN) has been implemented, for instance, in the personalized and mobile news access system. In this system, the kNN method was used to model the short-term interests of the users, which is of particular importance in the application domain of news recommendation. On arrival of a new message, the system looks for similar items in the set of stories that were recently rated by the user. By taking into account the last ratings only, the method is thus capable of adapting quickly and focusing on the user's short-term interests, which might be to read follow-up stories to recent events. At the same time, when relying on nearest Neighbors, it is also possible to set an upper threshold for item similarity to prevent the system from recommending items that the user most probably has already seen.

In the described system, the kNN method was implemented as part of a multistrategy user profile technique. The system-maintained profiles of short term (ephemeral) and long-term interests. The short-term profile, as described earlier, allows the system to provide the user with information on topics of recent interest. For the long-term model, the system described therefore collects information over a longer period of time (e.g., several months) and also seeks to identify the most informative words in the documents by determining the terms that consistently receive high TF-IDF scores in a larger document collection. The prediction of whether an item is of interest with respect to the long-term user model is based on a probabilistic classification technique,

Given the interest predictions and recommendations for the short-term and the long-term user models, the question remains how to combine them. In the described system, a rather simple strategy is chosen. Neighbors in the short-term model are searched; if no such Neighbors exist, the long-term user model is used. Other combinations are also possible. One option would be to acquire the short-term preferences online—for example, by questioning topics of interest and then sorting the matching items based on the information from the long-term preferences.

kNN-based methods have the advantage of being relatively simple to implement<sup>4</sup>, adapt quickly to recent changes, and have the advantage that, when compared with other learning approaches, a relatively small number of ratings is sufficient to make a prediction of reasonable quality. However, as experiments show, the prediction accuracy of pure kNN methods can be lower than those of other more sophisticated techniques.

Relevance feedback—Rocchio's method in content-based recommendation system

Relevance feedback using Rocchio's method can be effectively applied in content-based recommendation systems to improve the accuracy and relevance of item recommendations based on user feedback. Here's how relevance feedback within Rocchio's method can be adapted for content-based recommendation systems:

### Overview of Content-Based Recommendation Systems

In a content-based recommendation system, items (such as movies, products, articles) are represented by their features or attributes. The system recommends items similar to those that a user has liked or interacted with in the past.

### Application of Rocchio's Method for Relevance Feedback

#### 1. Item Representation:

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

- Each item  $i$  is represented as a feature vector  $\mathbf{i}$  in a high-dimensional space based on its attributes (e.g., genre, keywords, metadata).

### 2. User Profile Initialization:

- Initially, a user profile  $\mathbf{u}$  is initialized as a vector representing the user's preferences or interests. This profile can start as a zero vector or be based on known user preferences.

### 3. Item Recommendation:

- Recommend items by calculating the similarity between the user profile  $\mathbf{u}$  and each item  $\mathbf{i}$  using a similarity metric (e.g., cosine similarity). Items with higher similarity scores are considered for recommendation.

### 4. Recommendation Process

Initial Recommendations: Recommend items to the user based on the similarity between the user profile  $\mathbf{u}$  and item feature vectors  $\mathbf{i}$ .

### 5. Relevance Feedback Process using Rocchio's Method

#### User Interaction:

- Present recommended items to the user.
- Allow the user to provide feedback on the relevance or satisfaction with the recommended items. Feedback can include explicit ratings (e.g., thumbs up, thumbs down) or implicit signals (e.g., clicks, views).

#### Updating User Profile:

- Use Rocchio's method to update the user profile  $\mathbf{u}$  based on the received feedback:

$$\mathbf{u}^{\text{new}} = \alpha \cdot \mathbf{u} + \beta \cdot \frac{1}{|P|} \sum_{\mathbf{i} \in P} \mathbf{i}$$

For negatively rated items (or items not interacted with):

$$\mathbf{u}^{\text{new}} = \alpha \cdot \mathbf{u} - \gamma \cdot \frac{1}{|N|} \sum_{\mathbf{i} \in N} \mathbf{i}$$

where:

- $P$  is the set of positively rated items.
- $N$  is the set of negatively rated items or items not interacted with.
- $|P|$  and  $|N|$  are the number of positively and negatively rated items, respectively.
- $\alpha$ ,  $\beta$ , and  $\gamma$  are weighting factors controlling the influence of positive and negative feedback on the user profile updates.

### 6. Iterative Refinement:

- After updating the user profile based on the feedback, recompute item recommendations using the updated user profile.



## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

- Iterate this process: present new recommendations, gather feedback, update the user profile, and refine the recommendations until convergence or a stopping condition is met.

### Benefits and Considerations

- **Personalization:** Rocchio's method enhances the personalization of recommendations by adapting to individual user preferences over time.
- **Adaptability:** The system can quickly respond to changes in user preferences based on real-time feedback.
- **User Engagement:** By allowing users to influence recommendations, the system encourages user engagement and satisfaction.

Another method that is based on the vector-space model and was developed in the context of the pioneering information retrieval (IR) system SMART in the late 1960s is Rocchio's relevance feedback method. A particular aspect of SMART was that users could not only send (keyword-based) queries to the system but could also give feedback on whether the retrieved items were relevant. With the help of this feedback, the system could then internally extend the query to improve retrieval results in the next round of retrieval.

The SMART system for information retrieval did not exploit such additional information but rather allowed the user to interactively and explicitly rate the retrieved documents— that is, to tell the system whether they were relevant.

This information is subsequently used to further refine the retrieval results. The rationale of following this approach is that with pure query- and similarity based methods that do not provide any feedback mechanisms, the retrieval quality depends too strongly on the individual user's capability to formulate queries that contain the right keywords. User-defined queries often consist only of very few and probably polysemous words; a typical query on the web, for instance, consists of only two keywords on average.

The relevance feedback loop used in this method will help the system improve and automatically extend the query as follows. The main idea is to first split the already rated documents into two groups,  $D^+$  and  $D^-$ , of liked (interesting/relevant) and disliked documents and calculate a prototype (or average) vector for these categories. This prototype can also be seen as a sort of centroid of a cluster for relevant and nonrelevant document sets; see Figure 3.1.

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

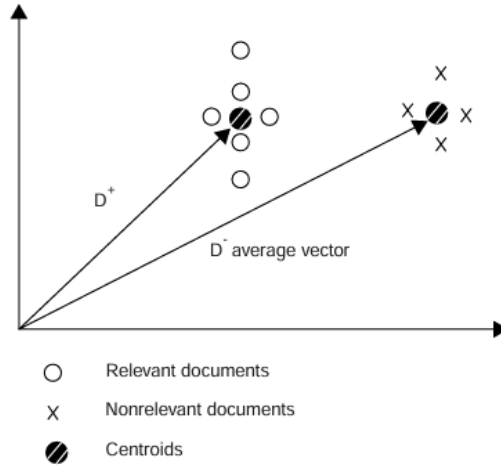


Figure 3.1. Average vectors for relevant and nonrelevant documents.

The current query  $Q_i$ , which is represented as a multidimensional term vector just like the documents, is then repeatedly refined to  $Q_{i+1}$  by a weighted addition of the prototype vector of the relevant documents and weighted subtraction of the vector representing the nonrelevant documents. As an effect, the query vector should consistently move toward the set of relevant documents as depicted schematically in Figure 3.2.

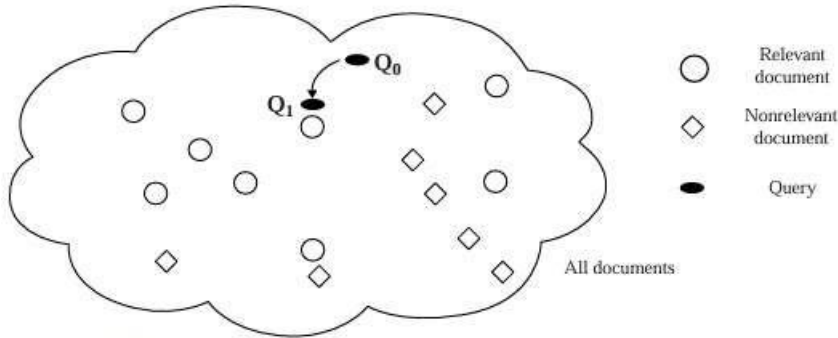


Figure 3.2. Relevance feedback. After feedback, the original query is moved toward the cluster of the relevant documents; see also Manning et al. (2008).

The proposed formula for computing the modified query  $Q_{i+1}$  from  $Q_i$  is defined as follows:

$$Q_{i+1} = \alpha * Q_i + \beta \left( \frac{1}{|D^+|} \sum_{d^+ \in D^+} d^+ \right) - \gamma \left( \frac{1}{|D^-|} \sum_{d^- \in D^-} d^- \right)$$

The variables  $\alpha$ ,  $\beta$ , and  $\gamma$  are used to fine-tune the behavior of the “move” toward the more relevant documents. The value of  $\alpha$  describes how strongly the last (or original) query should be weighted, and  $\beta$  and  $\gamma$  correspondingly capture how strongly positive and negative feedback should be taken into account in the improvement step.

The first point can be partially automated by trying to capture the user ratings implicitly— for instance, by interpreting a click on a proposed document as a positive rating. The question of

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

whether such assumptions hold in general what to do when a user has read an article but was disappointed and what other techniques for gathering implicit feedback can be used— remains open

Another technique for circumventing the problem of acquiring explicit user feedback is to rely on pseudo relevance feedback (blind feedback). The basic idea is to assume that the first  $n$  (say, 10) documents that match the query best with respect to the vector similarity measure are considered relevant. The set  $D^-$  is not used ( $\gamma$  is set to 0) unless an explicit negative feedback exists.

The second point— that user interaction is required during the proposal generation phase— at first glance appears to be a disadvantage over the fully automated proposal generation process of CF approaches. In fact, interactive query refinement also opens new opportunities for gathering information about the user's real preferences and may help the user to “learn” which vocabulary to use to retrieve documents that satisfy his or her information needs. The main assumption, of course, is that the user is capable of formulating a proper initial query, an assumption that might not always hold if we think of terminology aspects, multilanguage translation problems, or simply word misspellings

### **classification methods in content-based recommendation systems**

Content-based systems utilize various classification algorithms to effectively categorize and recommend items based on their features and attributes. These algorithms help in modeling item representations and making decisions about item similarities or relevance to users' preferences. Here are some common classification algorithms used in content-based systems:

#### 1. Vector Space Models (VSM)

**Description:** VSM represents items and user profiles as vectors in a high-dimensional space based on item features or user preferences.

**Application:** It's foundational for content-based systems to measure similarity between item vectors and user profiles using metrics like cosine similarity.

#### 2. k-Nearest Neighbors (k-NN)

**Description:** k-NN classifies items based on the majority class among their nearest neighbors in the feature space.

**Application:** In content-based systems, k-NN can be used to identify similar items or to find the most similar items to a user's preferences.

#### 3. Decision Trees

**Description:** Decision trees partition the feature space into a hierarchical structure of decisions and outcomes.

**Application:** They can be used in content-based systems to make decisions about item categories or attributes based on their features.

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

### 4. Random Forests

**Description:** Random forests are ensembles of decision trees that improve accuracy and robustness.

**Application:** They can be utilized in content-based systems for classification tasks where multiple decision trees collectively make predictions about item categories or similarities.

### 5. Support Vector Machines (SVM)

**Description:** SVMs find hyperplanes that best separate different classes in the feature space.

**Application:** In content-based systems, SVMs can be employed for item classification tasks, such as determining item categories or relevance to user preferences.

### 6. Naive Bayes

**Description:** Naive Bayes is based on Bayes' theorem and assumes independence among features.

**Application:** It's used in content-based systems for probabilistic classification tasks, such as text categorization or sentiment analysis of item descriptions.

### 7. Neural Networks

**Description:** Deep learning neural networks learn complex patterns from item features through multiple layers of neurons.

**Application:** In advanced content-based systems, neural networks can be used for feature learning, item representation, and classification tasks, achieving high accuracy and scalability.

### 8. Clustering Algorithms (e.g., k-Means)

**Description:** Clustering algorithms group items based on similarity into clusters.

**Application:** They can be applied in content-based systems for unsupervised learning tasks, such as identifying item categories or user preferences based on item features.

### 9. Ensemble Methods

**Description:** Ensemble methods combine multiple classifiers to improve overall performance and robustness.

**Application:** They can be used in content-based systems to aggregate predictions from multiple classifiers, enhancing recommendation accuracy and diversity.

### 10. Reinforcement Learning

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

**Description:** Reinforcement learning algorithms learn to make sequential decisions based on rewards received from the environment.

**Application:** In interactive content-based systems, reinforcement learning can optimize item recommendations over time based on user feedback and interactions.

Content-based systems leverage a combination of these classification algorithms to model item representations, learn user preferences, and make decisions about item similarity or relevance. The choice of algorithm depends on the specific use case, the nature of item features, and the desired level of recommendation accuracy and scalability. Advanced systems often employ a mix of traditional machine learning algorithms and deep learning techniques to achieve optimal performance in content-based recommendation tasks.

### Text classification methods used in content-based recommendation systems

Text classification methods play a crucial role in content-based recommendation systems, especially when dealing with textual data such as product descriptions, article content, or user reviews. These methods enable the system to understand and categorize textual content, which is essential for accurate item representation and personalized recommendations. Here are some text classification methods commonly used in content-based recommendation systems:

#### 1. Term Frequency-Inverse Document Frequency (TF-IDF)

- **Description:** TF-IDF calculates the importance of each word (term) in a document relative to a collection of documents. It represents documents as vectors based on the frequency of terms, adjusted by their rarity across the corpus.
- **Application:** TF-IDF is used to transform textual data into numerical representations suitable for machine learning algorithms like cosine similarity, enabling content-based systems to measure similarity between items and user profiles.

#### 2. Word Embeddings (e.g., Word2Vec, GloVe)

- **Description:** Word embeddings represent words as dense vectors in a continuous vector space, capturing semantic relationships between words.
- **Application:** In content-based systems, pre-trained word embeddings can be used to convert text into numerical vectors, providing rich representations for item features and enhancing classification accuracy.

#### 3. Text Vectorization (Bag-of-Words, n-grams)

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

- **Description:** Text vectorization methods convert text into numerical representations based on word frequencies (Bag-of-Words) or sequences of words (n-grams).
- **Application:** These methods are foundational for text classification in content-based systems, allowing algorithms to process and classify textual data efficiently.

### 4. Naive Bayes Classifier

- **Description:** Naive Bayes is a probabilistic classifier based on Bayes' theorem with an assumption of independence between features.
- **Application:** It's used for text classification tasks in content-based systems, such as categorizing items or reviews into predefined classes (e.g., product categories, sentiment labels).

### 5. Support Vector Machines (SVM)

- **Description:** SVMs find the hyperplane that best separates different classes in the feature space.
- **Application:** In content-based recommendation systems, SVMs can be used for text classification tasks, such as assigning categories or relevance scores to items based on textual features.

### 6. Deep Learning Models (e.g., Convolutional Neural Networks, Recurrent Neural Networks)

- **Description:** Deep learning models learn hierarchical representations of textual data through multiple layers of neurons.
- **Application:** These models are effective for text classification in content-based systems, capable of capturing complex patterns and relationships in textual content to improve recommendation accuracy.

### 7. Ensemble Methods

- **Description:** Ensemble methods combine multiple classifiers to improve prediction accuracy and robustness.
- **Application:** In content-based recommendation systems, ensemble methods can be used to aggregate predictions from multiple text classifiers, enhancing the system's ability to classify items and generate personalized recommendations.

### 8. Topic Modeling (e.g., Latent Dirichlet Allocation, LDA)

- **Description:** Topic modeling algorithms identify latent topics in a collection of documents and assign topics to individual documents based on word distributions.

## UNIT II CONTENT-BASED RECOMMENDATION SYSTEMS

- **Application:** Topic modeling can be used in content-based systems to discover underlying themes or topics in textual content, aiding in item categorization and recommendation based on thematic relevance.

### 9. Sequence Models (e.g., Long Short-Term Memory, LSTM)

- **Description:** Sequence models are effective for processing sequential data (e.g., text, time-series).
- **Application:** In content-based recommendation systems, sequence models can be used to classify and analyze sequential textual data, such as user interactions or item descriptions, to generate personalized recommendations.

Text classification methods are fundamental to content-based recommendation systems, enabling the systems to process, understand, and classify textual data for effective item representation and recommendation. The choice of text classification method depends on the specific use case, the nature of textual data, and the desired level of recommendation accuracy and personalization. Advanced systems often leverage a combination of these methods, along with other machine learning techniques, to achieve optimal performance in content-based recommendation tasks.